

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 3: Iteration and Tracing

Learning Outcomes

- Understand iteration in Java using while and for loops.
- Learn to trace variable values during iteration.

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

Some of the instructions may be executed more than once

Introduction to Iteration

Iteration: A process of repeating a set of statements until a condition is met.

Common loop structures in Java:

- `for` loop
- `while` loop

Why use loops?

- Reduce code repetition
- Handle repetitive tasks like traversing data

While Loop Syntax

```
while (condition) {  
    // Statements to execute  
}
```

While loop Example

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++; // Increment
}
```

Trace

- Before loop: `i = 0`
- Iteration 1: `i = 1`
- Iteration 2: `i = 2`
- ... (until `i < 5` becomes `false`)

For Loops

For Loop Syntax

```
for (initialization; condition; increment) {  
    // Statements to execute  
}
```

Example

```
for (int n = 1; n <= 10; n = n + 1) {  
    // Statements to execute  
}
```

For Loops

For loops are good for repeating things a bounded number of times.

We can count up:

```
// Count up from 1 to 10.  
System.out.println("I'm angry.");  
for (int n = 1; n <= 10; n = n + 1) {  
    System.out.println(n);  
}  
System.out.println("Now I feel calmer.");
```

We can also count down:

```
// Count down from 10 to 1.  
System.out.println("Prepare for launch.");  
for (int n = 10; n >= 1; n = n - 1) {  
    System.out.println(n);  
}  
System.out.println("Lift off!");
```

Breaking down a for loop

Make sure you get the **end-point** of the loop correct.

The two code bellow has **two different output**

```
System.out.println("I'm angry.");  
for (int n = 1; n <= 10; n = n + 1) {  
    System.out.print("X");  
}
```

```
System.out.println("I'm angry.");  
for (int n = 1; n < 10; n = n + 1) {  
    System.out.print("X");  
}
```

Breaking down a for loop

Make sure you get the **end-point** of the loop correct.

The two code bellow has **two the same output**

```
System.out.println("I'm angry.");  
for (int n = 1; n <= 10; n = n + 1) {  
    System.out.print("X");  
}
```

```
System.out.println("I'm angry.");  
for (int n = 0; n < 10; n = n + 1) {  
    System.out.print("X");  
}
```

Increment and Decrement Operators

Adding and subtracting 1 from a variable are so common that there are special **increment** (`++`) and **decrement** (`--`) operators for convenience.

```
System.out.println("I'm angry.");  
for (int n = 1; n <= 10; n++) {  
    System.out.println(n);  
}  
System.out.println("Now I feel calmer.");
```

```
System.out.println("Prepare for launch.");  
for (int n = 10; n > 0; n--) {  
    System.out.println(n);  
}  
System.out.println("Lift-off!");
```

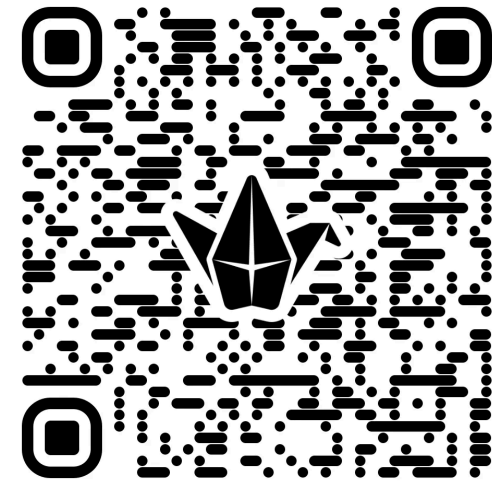
(You can also write `++n` and `--n`. Is there a difference?)

Quiz 1 ([click here](#) or scan QRCode)

Which is the output of the following code?

```
for (int n = 2; n <= 8; n = n + 2) {  
    System.out.print(n + " ");  
}
```

- a. 2 3 4 5 6 7 8
- b. 2 4 6
- c. 2 4 6 8
- d. 2 4 6 8 10



Student Loan Revisited

```
double loan = 13348;  
double interest = 7.3;  
double compound = 1 + (interest/100);  
double owed = loan * compound;  
owed = (owed + loan) * compound;  
owed = (owed + loan) * compound;  
System.out.printf("In 3 years, your loan will be %f pounds.%n", owed);
```

Student Loan Revisited using loop

```
double loan = 13348;
double interest = 7.3;
double compound = 1 + (interest/100);
int years=3;
double owed = loan * compound;
for (int i=0; i < years-1; i++)
    owed = (owed + loan) * compound;
System.out.printf("In %d years, your loan will be %f pounds.\n", years, owed);
```

Comparing while and for

One way to see is `while` and `loop` statements is looking to its bounds

While Loops — Unbounded (≥ 0) Iteration

- A `while` loop continues to iterate as long as the condition is `true`.
- The number of iterations unbounded (≥ 0).

Example

```
int i = 0;
while (i < 5) {
    System.out.println("i = " + i);
    i++;
}
```

For Loops — Unbounded (≥ 0) Iteration

- A `for` loop continues to iterate as based in the increment
- The number of iterations unbounded (≥ 0).

Example

```
int i = 0;
while (i < 5) {
    System.out.println("i = " + i);
    i++;
}
```

Equivalence of For and While Loops

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
    i++;  
}
```

Unbounded (≥ 1) Iteration — Do-While Loops

- The `do-while` loop guarantees at least one iteration.

Syntax

```
do {  
    // Statements  
} while (condition);
```

Example

```
int i = 0;  
do {  
    System.out.println(i);  
    i++;  
} while (i < 5);
```

Loop tracing

Tracing Variables in a While Loop

Example

```
int x = 3;  
while (x > 0) {  
    x--;  
}
```

Trace just after the line `x--`

Iteration	x Value
1	2
2	1
3	0

Tracing Variables in a For Loop

Example

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

Trace just after the line `println`

Iteration	i Value
1	0
2	1
3	2
4	3
5	4

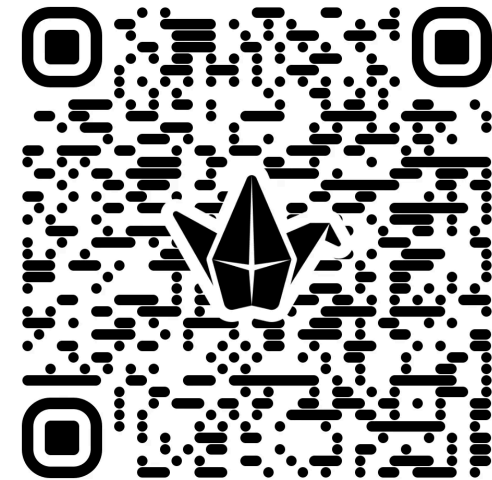
Tracing variable during a loop execution helps understanding some unexpected behaviour

Quiz 2 ([click here](#) or scan QRCode)

The following Java code is intended to compute the integer square root, but which is its output?

```
int n = 10;  
int sqrt = 0;  
while (sqrt * sqrt < n) {  
    sqrt++;  
}  
System.out.println("Integer square root: " + sqrt);
```

- a. 4
- b. 3
- c. 1
- d. 10



```
int n = 10;  
int sqrt = 0;  
  
while (sqrt * sqrt < n) {  
    sqrt++;  
    System.out.println("Integer square root: " + sqrt);  
}
```

- `n = 10`, the expected integer square root is 3
- Issue: The program outputs 4 instead of 3.

Example

```
int n = 10;  
int sqrt = 0;  
  
while (sqrt * sqrt < n) {  
    sqrt++;  
    System.out.println("Integer square root: " + sqrt);  
}
```

Trace just after the line `sqrt++;`

Iteration	sqrt	sqrt*sqrt	sqrt * sqrt < n
1	1	1	true
2	2	4	true
3	3	9	true
4	4	16	false

Nested Loops and Breaks

Nesting For Loops

Syntax

A `for` loop can be placed inside another `for` loop. This is called **nested loops**.

```
for (int i = 0; i < MAX_I; i++) {  
    for (int j = 0; j < MAX_J; j++) {  
        // Inner loop code  
    }  
}
```

Nested Loops

Example

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 2; j++) {  
        System.out.print("i=" + i);  
        System.out.println(",j=" + j);  
    }  
}
```

Output

```
i=1,j=1  
i=1,j=2  
i=2,j=1  
i=2,j=2  
i=3,j=1  
i=3,j=2
```

Break statement

- The `break` statement exits the loop immediately when encountered.
- Useful for exiting early from loops.

Example

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // Exit the loop when i is 5  
    }  
    System.out.println(i);  
}
```

Break in a nested loop

Example

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 2; j++) {  
        if (i == 2 && j == 2) {  
            break;  
        }  
        System.out.print("i=" + i);  
        System.out.println(",j=" + j);  
    }  
}
```

Output

```
i=1,j=1  
i=1,j=2  
i=1,j=3  
i=2,j=1  
i=3,j=1  
i=3,j=2  
i=3,j=3
```

- Why the loop run when i is 3?

Break in a nested loop

One break for each for

Output

```
for (int i = 1; i <= 3; i++){  
    int j; // allow be used in if  
    for (j = 1; j <= 2; j++){  
        if (i == 2 && j == 2){  
            break;  
        }  
        System.out.print("i=" + i);  
        System.out.println(",j=" + j);  
    }  
    if (i == 2 && j == 2){  
        break;  
    }  
}
```

```
i=1,j=1  
i=1,j=2  
i=2,j=1
```

Elaborated Loops

Snakes and Ladders

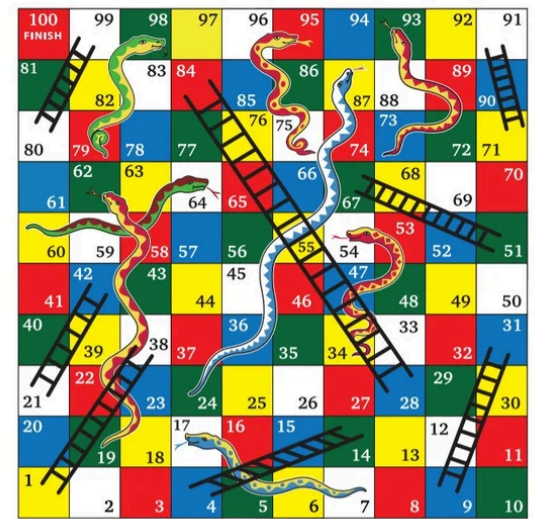
“Snakes and ladders” is a popular children's board game, played on a grid of numbered squares with drawings of snakes and ladders overlaid.

Each player has a counter, which starts off the board, next to square 1.

Players take it in turns to roll the dice and move their counters forward that many squares.

If your counter lands on a square with the head of a snake, it immediately slides down to the tail of the snake.

If your counter lands on a square with the foot of a ladder, it immediately climbs up to the top of the ladder.



Snakes and Ladders

Problem: Write a computer game of Snakes and Ladders for one player.

```
final int WINNING_POSITION = 16; // 4x4 board, so max position is 16
Scanner scanner = new Scanner(System.in);
int playerPosition = 0;
Random dice = new Random();
boolean gameOver = false;
```

```
while (!gameOver) {  
    System.out.println("Your turn.");  
    System.out.print("Press Enter to roll the dice...");  
    scanner.nextLine();  
  
    int diceRoll = dice.nextInt(6) + 1; // Roll a dice (1-6)  
    System.out.println("You rolled a " + diceRoll);  
  
    int newPosition = playerPosition + diceRoll;  
  
    ...  
}
```

```
if (newPosition > WINNING_POSITION) {
    System.out.println("You rolled too high and stayed at " + playerPosition);
} else {
    playerPosition = newPosition;
    // Handle ladders
    if (playerPosition == 3) {
        System.out.println("You found a ladder! Climb up to 11.");
        playerPosition = 11;
    } else if (playerPosition == 5) {
        System.out.println("You found a ladder! Climb up to 9.");
        playerPosition = 9;
    }
    // Handle snakes
    if (playerPosition == 15) {
        System.out.println("Oops! You hit a snake! Slide down to 7.");
        playerPosition = 7;
    } else if (playerPosition == 12) {
        System.out.println("Oops! You hit a snake! Slide down to 2.");
        playerPosition = 2;
    }
    System.out.println("You are now at position " + playerPosition);
}
```

```
// Check for win condition
if (playerPosition == WINNING_POSITION) {
    System.out.println("Congratulations! You won the game!");
    gameOver = true;
}
}
```

Bouncing Ball

Problem: Write a programming that display a ball baling. Consider that the ball has:

- `acceleration` : downward acceleration (simulating gravity), which decreases the height over time.
- `elasticity` : how the ball behaves when it hits the ground.
- `rows` : Represents the total number of rows in the output that simulate the height of the bouncing ball.

Help full characters for screen commands

Code	Effect
"\033[2J"	Clear the screen.
"\033[?25l"	Hide the cursor.
"\033[H"	Move the cursor to the upper-left corner of the screen.

See other [here](#).

Bouncing Ball

```
double height = 20.0;
double velocity = 0.0;
double acceleration = -0.1;
double elasticity = 0.85;
int rows = 25;

String erase = "\033[2J";
String hide = "\033[?251";
String home = "\033[H";

System.out.print(erase);
System.out.print(hide);
```

```
for (int frame = 0; frame < 500; frame++) {  
    height = height + velocity;  
    if (height <= 0.0) {  
        height = height - velocity;  
        velocity = velocity * -1.0 * elasticity;  
    }  
    else {  
        velocity = velocity + acceleration;  
    }  
  
    System.out.print(home);  
    int row = rows - (int) height;  
    for (int y = 1 ; y <= rows; y++) {  
        if (y == row) {  
            System.out.println("O");  
        }  
        else {  
            System.out.println(" ");  
        }  
    }  
    Thread.sleep(40);  
}
```

Review Learning Outcomes

Review Learning Outcomes

1. Understand iteration in Java using while and for loops.

- Loops (while/for) allow create repetitive tasks.

2. Learn to trace variable values during iteration.

- Tracing helps understand how variables change during iterations.

DEPARTMENT OF COMPUTER SCIENCE



Thank you! 😊

Do you have any questions 🙋 ?

The next lecture will cover: Functions